

MACHINE LEARNING IN FINANCE

MODULE 7 | LESSON 3

MODEL EVALUATION AND REGULARIZATION

Reading Time	40 minutes
Prior Knowledge	Introduction to hyperparameter tuning, loss functions
Keywords	simulated annealing

In the previous lesson, we applied grid search and random search hyperparameter tuning techniques to a house prediction problem and evaluated the model performance. In this lesson, we will present insights on when to use different cross-validation strategies. We will also discuss model evaluation metrics and introduce the notion of having a baseline model to compare with our model performance.

1. Model Evaluation

In the previous modules, we have encountered different learning algorithms, and in all of them, we are interested in minimizing errors when presented with never-before-seen data. To achieve this, we will discuss the following concepts in this section:

1. Choosing an appropriate cross-validation strategy.
2. Building a baseline model that acts as a reference for the actual model.
3. Nested cross-validation concepts and their evaluation.
4. Metrics used in classification and regression models.

1.1 Building Baseline Models

The baseline model can be thought of as a model reference to the actual model. We should consider the following points when building a baseline model:

1. It should be a simple model that is used for comparison.
2. It should be easy to explain.
3. It should be based on the actual data.

Baseline models are important for the following reasons;

Understanding the Data

When we train a baseline model on our dataset, we can infer the following:

- A baseline model where prediction of our target variable is completely off indicates how difficult it will be for weaker models to get patterns in the dataset.
- It can help us point out the segment of the data that gives the model a higher error.

Faster Modeling

Once we have a baseline model in place, building the actual model pipeline becomes easier as we understand the data better.

Performance Benchmark

The performance of the baseline model help us to decide whether we will need a complex model or a simple one.

We can divide the baseline model into two types:

1. Simple baseline model.
2. Machine learning baseline model.

In this subsection, we will consider a simple baseline model that uses simple logic to build the baseline model.

```
In [ ]: from sklearn.datasets import fetch_california_housing

data, target = fetch_california_housing(return_X_y=True, as_frame=True)
```

We split the data using `ShuffleSplit` with 20% of the data used for validation.

```
In [ ]: from sklearn.model_selection import ShuffleSplit

cv = ShuffleSplit(n_splits=30, test_size=0.2, random_state=0)
```

We define a decision tree regression function as our model for this section.

```
In [ ]: import pandas as pd
from sklearn.model_selection import cross_validate
from sklearn.tree import DecisionTreeRegressor

regressor = DecisionTreeRegressor()
cv_results_tree_regressor = cross_validate(
    regressor, data, target, cv=cv, scoring="neg_mean_absolute_error", n_jobs=2
)

errors_tree_regressor = pd.Series(
    -cv_results_tree_regressor["test_score"], name="Decision tree regressor"
)
```

We use a dummy regressor as our baseline that predicts the mean of the target computed.

```
In [ ]: from sklearn.dummy import DummyRegressor
```

```

dummy = DummyRegressor(strategy="mean")
result_dummy = cross_validate(
    dummy, data, target, cv=cv, scoring="neg_mean_absolute_error", n_jobs=2
)
errors_dummy_regressor = pd.Series(-result_dummy["test_score"], name="Dummy regr
errors_dummy_regressor.describe()

```

In the next cell, we prepare a table that compares the performance of the decision tree regressor and the dummy regressor.

```

In [ ]: all_errors = pd.concat(
    [errors_tree_regressor, errors_dummy_regressor],
    axis=1,
)
all_errors

```

We see from the table above that in all the splits, the decision tree regressor has fewer errors compared to the dummy regressor. In the next cell, we can visualize the distribution of errors for both regression functions.

```

In [ ]: import matplotlib.pyplot as plt
import numpy as np

bins = np.linspace(start=0, stop=1, num=80)
all_errors.plot.hist(bins=bins, edgecolor="black")
plt.legend(bbox_to_anchor=(1.05, 0.8), loc="upper left")
plt.xlabel("Mean absolute error")
plt.suptitle(
    "Fig. 1: Cross-validation testing errors.",
    fontweight="bold",
    horizontalalignment="right",
)
plt.show()

```

1.2 Choosing a Cross-Validation Strategy

So far we have only used `ShuffleSplit` or `KFold` to perform cross-validation, but they may not work all the time. As such, it's important to choose a cross-validation strategy wisely, depending on the task at hand. Let's consider the example below to demonstrate why it is important to choose a cross-validation strategy carefully.

```

In [ ]: from sklearn.datasets import load_iris

data, target = load_iris(as_frame=True, return_X_y=True)

```

We now create a simple classification algorithm for this task.

```

In [ ]: from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

model = make_pipeline(StandardScaler(), LogisticRegression())

```

We now use this model to train and generalize on unseen data.

```
In [ ]: from sklearn.model_selection import KFold, cross_validate

cv = KFold(n_splits=3)
results = cross_validate(model, data, target, cv=cv)
test_score = results["test_score"]
print(f"The average accuracy is " f"{test_score.mean():.3f} ± {test_score.std():
```

The output above shows that the model was unable to get any prediction correct when exposed to validation data. To understand this peculiar output, we investigate the target feature.

```
In [ ]: import matplotlib.pyplot as plt

target.plot()
plt.xlabel("Sample index")
plt.ylabel("Class")
plt.yticks(target.unique())
plt.suptitle(
    "Fig. 2: Class value in target y.", fontweight="bold", horizontalalignment="
)
plt.show()
```

We can observe from the above diagram that the target column is ordered, and this causes an unexpected outcome when we use cross-validation. We will now compute the class counts for the training and validation sets in the cells below.

```
In [ ]: import pandas as pd

n_splits = 3
cv = KFold(n_splits=n_splits)

train_cv_counts = []
test_cv_counts = []
for fold_idx, (train_idx, test_idx) in enumerate(cv.split(data, target)):
    target_train, target_test = target.iloc[train_idx], target.iloc[test_idx]

    train_cv_counts.append(target_train.value_counts())
    test_cv_counts.append(target_test.value_counts())
```

We display this in the table below.

```
In [ ]: train_cv_counts = pd.concat(
    train_cv_counts, axis=1, keys=[f"Fold #{idx}" for idx in range(n_splits)]
)
train_cv_counts.index.name = "Class label"
train_cv_counts
```

We can see that when training the model, in each fold, only two of the classes are available, and thus, it becomes difficult to predict on unseen data.

```
In [ ]: test_cv_counts = pd.concat(
    test_cv_counts, axis=1, keys=[f"Fold #{idx}" for idx in range(n_splits)]
)
test_cv_counts.index.name = "Class label"
test_cv_counts
```

In the test data, only one class, which was not available in training, is used. This makes it difficult to predict, as the class was not available when training. The cells below help visualize the above tables.

```
In [ ]: train_cv_counts.plot.bar()
plt.legend(bbox_to_anchor=(1.05, 0.8), loc="upper left")
plt.ylabel("Count")
plt.suptitle(
    "Fig. 3: Training Set Data Distribution in Folds.",
    fontweight="bold",
    horizontalalignment="right",
)
plt.show()
```

```
In [ ]: test_cv_counts.plot.bar()
plt.legend(bbox_to_anchor=(1.05, 0.8), loc="upper left")
plt.ylabel("Count")
plt.suptitle(
    "Fig. 4: Data distribution in Testing Set",
    fontweight="bold",
    horizontalalignment="right",
)
plt.show()
```

To solve the problems above, it is a good practice to shuffle our dataset before we start the modeling phase as shown below.

```
In [ ]: cv = KFold(n_splits=3, shuffle=True, random_state=0)
results = cross_validate(model, data, target, cv=cv)
test_score = results["test_score"]
print(f"The average accuracy is " f"{test_score.mean():.3f} ± {test_score.std():.3f}")
```

We can now see that the model accuracy has really improved to more than 95%. We can now visualize the count of class in each fold for both the training and testing samples.

```
In [ ]: train_cv_counts = []
test_cv_counts = []
for fold_idx, (train_idx, test_idx) in enumerate(cv.split(data, target)):
    target_train, target_test = target.iloc[train_idx], target.iloc[test_idx]

    train_cv_counts.append(target_train.value_counts())
    test_cv_counts.append(target_test.value_counts())
train_cv_counts = pd.concat(
    train_cv_counts, axis=1, keys=[f"Fold #{idx}" for idx in range(n_splits)]
)
test_cv_counts = pd.concat(
    test_cv_counts, axis=1, keys=[f"Fold #{idx}" for idx in range(n_splits)]
)
train_cv_counts.index.name = "Class label"
test_cv_counts.index.name = "Class label"
```

```
In [ ]: train_cv_counts.plot.bar()
plt.legend(bbox_to_anchor=(1.05, 0.8), loc="upper left")
plt.ylabel("Count")
plt.suptitle(
```

```

    "Fig. 5: Training set Data Distribution (kFold).",
    fontweight="bold",
    horizontalalignment="right",
)
plt.show()

```

From the bar plots above, we can observe that the classes are fairly distributed in each fold. This is also true on the testing dataset below.

```

In [ ]: test_cv_counts.plot.bar()
plt.legend(bbox_to_anchor=(1.05, 0.8), loc="upper left")
plt.ylabel("Count")
plt.suptitle(
    "Fig. 6: Testing set Data Distribution (kFold).",
    fontweight="bold",
    horizontalalignment="right",
)
plt.show()

```

Another technique will involve splitting our dataset so as to preserve the original class counts in each validation split. This is done using stratification as shown in the cells below. The steps will remain as what we have seen in the two strategies before.

```

In [ ]: from sklearn.model_selection import StratifiedKFold

cv = StratifiedKFold(n_splits=3)

```

```

In [ ]: results = cross_validate(model, data, target, cv=cv)
test_score = results["test_score"]
print(f"The average accuracy is " f"{test_score.mean():.3f} ± {test_score.std():.3f}")

```

```

In [ ]: train_cv_counts = []
test_cv_counts = []
for fold_idx, (train_idx, test_idx) in enumerate(cv.split(data, target)):
    target_train, target_test = target.iloc[train_idx], target.iloc[test_idx]

    train_cv_counts.append(target_train.value_counts())
    test_cv_counts.append(target_test.value_counts())
train_cv_counts = pd.concat(
    train_cv_counts, axis=1, keys=[f"Fold #{idx}" for idx in range(n_splits)]
)
test_cv_counts = pd.concat(
    test_cv_counts, axis=1, keys=[f"Fold #{idx}" for idx in range(n_splits)]
)
train_cv_counts.index.name = "Class label"
test_cv_counts.index.name = "Class label"

```

```

In [ ]: train_cv_counts.plot.bar()
plt.legend(bbox_to_anchor=(1.05, 0.8), loc="upper left")
plt.ylabel("Count")
plt.suptitle(
    "Fig. 7: Training set Data Distribution (Stratified).",
    fontweight="bold",
    horizontalalignment="right",
)
plt.show()

```

```
In [ ]: test_cv_counts.plot.bar()
plt.legend(bbox_to_anchor=(1.05, 0.8), loc="upper left")
plt.ylabel("Count")
plt.suptitle(
    "Fig. 8: Testing set Data Distribution (Stratified).",
    fontweight="bold",
    horizontalalignment="right",
)
plt.show()
```

We can see that the counts in the training and testing sets are not too far from each other.

1.3 Nested Cross-Validation

The k-fold cross validation technique that we have discussed so far is important for hyperparameter optimization and model evaluation on the validation set. The fact that we use cross-validation on the same dataset to tune our hyperparameters and choose a model from the same process may cause the model evaluation to be biased.

A work-around to this limitation is to create an outer loop of cross-validation. This process is called **nested cross-validation** and we will demonstrate this in the cells below.

We then configure the cross-validation process below. We will use the make classification dataset.

```
In [ ]: from numpy import mean, std
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import GridSearchCV, KFold, cross_val_score
```

We then split the dataset into a train and test dataset where the train dataset will also be used to train and validate the model.

```
In [ ]: from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    data, target, shuffle=True, random_state=0, test_size=0.3
)
```

We then define the model to be used for this problem.

```
In [ ]: model = RandomForestClassifier(random_state=1)
```

We now configure the inner cross-validation that will be used to train the hyperparameters.

```
In [ ]: cv_inner = KFold(n_splits=3, shuffle=True, random_state=1)
```

As we have seen from Lesson 1, we define the search space for our hyperparameters to be tuned and apply grid search to get the best combination of hyperparameters.

```
In [ ]: space = dict()
space["n_estimators"] = [10, 100, 500]
space["max_features"] = [2, 4, 6]
# Search for the best hyperparameter combination
search = GridSearchCV(
    model, space, scoring="accuracy", n_jobs=1, cv=cv_inner, refit=True
)
```

We now configure the cross-validation that will help us evaluate the model.

```
In [ ]: cv_outer = KFold(n_splits=10, shuffle=True, random_state=1)
```

The next cell performs the nested cross validation on our data and we output the result.

```
In [ ]: # execute the nested cross-validation
scores = cross_val_score(
    search, X_train, y_train, scoring="accuracy", cv=cv_outer, n_jobs=-1
)
# report performance
print("Accuracy: %.3f (%.3f)" % (mean(scores), std(scores)))
```

We get the average model accuracy from the 10 iterations, and the result shows that our model has a better accuracy. But in classification, we need to go beyond accuracy as a measurement of model performance. In the next section, we learn about classification metrics.

2. Metrics in Machine Learning

When building machine learning models, we need to measure their performance, and understanding the type of metric to use is vital if we are to judge whether we are making progress or not. As we are all aware now, machine learning tasks can be divided into regression or classification, and they are treated differently when modeling and also when measuring their performance. We will start by looking at regression metrics.

2.1 Regression Metrics

Linear regression involves finding the linear relationship that exists between the independent variables and the target variable, that is, $y_i(x_i, w) = w_0 + w_1 x_{i1} + w_2 x_{i2} + \dots + w_m x_{im} + \epsilon_i$ where $y_i \in \mathbb{R}$ is the target variable, $x_i \in \mathbb{R}^m$ is the predictor variables and ϵ_i is the random error. The random error occurs as a result of one of the following

- Choosing a wrong linear model.
- Selecting less relevant variables and omitting relevant ones.
- Measurement errors.

A good linear regression estimates the coefficient of the features, w_i , and minimizes the residual errors, $e_i = y_i - \hat{y}_i$. This is possible by minimizing the objective function. The objective function can take one of the forms below.

2.1.1 Mean Squared Error (MSE)

The mean squared error is given by $L = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$

2.1.2 Mean Absolute Error (MAE)

The mean absolute error is given by $L = \frac{1}{n} \sum_{i=1}^n |\hat{y}_i - y_i|$

2.1.3 Mean Squared Logarithmic Error

This is similar to MSE, but the dependent variable is in logarithmic scale: $L = \frac{1}{n} \sum_{i=1}^n (\log(\hat{y}_i + 1) - \log(y_i + 1))^2$

In the next cells, we will look at different objective functions in regression for measuring errors.

```
In [ ]: from sklearn.model_selection import train_test_split

data_train, data_test, target_train, target_test = train_test_split(
    data, target, shuffle=True, random_state=0
)
```

We first train the data using a decision-tree regressor and then evaluate on the test dataset. In the next cells, we look at the various metrics used to evaluate machine learning models. We start with most commonly used metric, the mean squared error.

```
In [ ]: from sklearn.metrics import mean_squared_error

regressor.fit(data_train, target_train)
target_predicted = regressor.predict(data_train)

print(
    f"Mean squared error on the training set: "
    f"{mean_squared_error(target_train, target_predicted):.3f}"
)
```

We measure the mean squared error on the test data below.

```
In [ ]: target_predicted = regressor.predict(data_test)

print(
    f"Mean squared error on the testing set: "
    f"{mean_squared_error(target_test, target_predicted):.3f}"
)
```

To calculate the coefficient of determination, R^2 , we calculate the `score()` function as shown below.

```
In [ ]: regressor.score(data_test, target_test)
```

We can also calculate the mean absolute error for our model as shown below.

```
In [ ]: from sklearn.metrics import mean_absolute_error

target_predicted = regressor.predict(data_test)
```

```
print(
    f"Mean absolute error: " f"{mean_absolute_error(target_test, target_predicted)
)
```

The median absolute error is equally calculated.

```
In [ ]: from sklearn.metrics import median_absolute_error

print(
    f"Median absolute error: "
    f"{median_absolute_error(target_test, target_predicted):.3f} "
)
```

2.1.4 Regularization

Regularization techniques are used in instances where a regression model overfits. The objective of regularization is to add a penalty term to the objective function that regularizes the size of the parameters, that is,

$$L = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2 + \lambda R(\vec{w})$$
 where $R(\vec{w})$ is the penalty term, a hyperparameter.

The most commonly used regularization techniques are ridge regression and lasso regression.

1. Ridge Regression

Ridge regression adds an L_2 -norm term to the objective function. The term causes weight decay to the variables. The loss function of a ridge regression is given by
$$L = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2 + \frac{\lambda}{2} \|\vec{w}\|^2$$

The λ hyperparameter controls the amount of decay where larger values of λ increase decay and thus make the model more robust to multi-collinearity. Note that for the value of $\lambda = 0$, ridge regression is equivalent to linear regression. An increase in the value of λ makes the coefficients of our model approach zero.

We start by importing the libraries for this demonstration.

```
In [ ]: import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
from sklearn.metrics import mean_squared_error

sns.set_style("darkgrid")
```

We import and split the dataset into train and test sets.

```
In [ ]: from sklearn.datasets import fetch_california_housing

X, y = fetch_california_housing(as_frame=True, return_X_y=True)
X.head()
```

```

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=25
)

```

We build the ridge model below and fit it to the training set.

```

In [ ]: from sklearn.linear_model import Ridge
        from sklearn.pipeline import Pipeline
        from sklearn.preprocessing import PolynomialFeatures, StandardScaler

        steps = [
            ("scalar", StandardScaler()),
            ("poly", PolynomialFeatures(degree=2)),
            ("model", Ridge()),
        ]

        ridge_pipe = Pipeline(steps)
        ridge_pipe.fit(X_train, y_train)
        # Predicting the Test set results
        y_pred = ridge_pipe.predict(X_test)

```

We apply k-Fold cross-validation on the dataset.

```

In [ ]: from sklearn.model_selection import cross_val_score

        accuracies = cross_val_score(estimator=ridge_pipe, X=X_train, y=y_train, cv=10)
        print("accuracy mean", accuracies.mean())
        print("accuracy standard deviation", accuracies.std())

```

We apply grid search to find the best model and the best alpha parameter.

```

In [ ]: from sklearn.metrics import make_scorer
        from sklearn.model_selection import GridSearchCV

        parameters = [{"model__alpha": np.arange(0, 0.2, 0.01)}]

        scoring_func = make_scorer(mean_squared_error)
        grid_search = GridSearchCV(
            estimator=ridge_pipe, param_grid=parameters, scoring=scoring_func, cv=10, n_
        )
        grid_search = grid_search.fit(X_train, y_train)

```

In the cell below, we display the alpha values that we intend to tune.

```

In [ ]: grid_search.cv_results_["params"]

```

As discussed in the model evaluation section, we will evaluate the performance of ridge regression on the dataset.

```

In [ ]: from sklearn.linear_model import Lasso, LinearRegression
        from sklearn.model_selection import (
            GridSearchCV,
            KFold,
            RepeatedKFold,
            cross_validate,

```

```

    train_test_split,
)
from sklearn.preprocessing import PolynomialFeatures, StandardScaler

```

We apply linear regression in the dataset and generate prediction on the test set.

```

In [ ]: lreg = LinearRegression()
lreg.fit(X_train, y_train)

lreg_y_pred = lreg.predict(X_test)

# calculating Mean Squared Error (MSE)
mean_squared_error = np.mean((lreg_y_pred - y_test) ** 2)
print("Mean squared Error on test set : ", mean_squared_error)

# Putting together the coefficient and their corresponding variable names
lreg_coefficient = pd.DataFrame()
lreg_coefficient["Columns"] = X_train.columns
lreg_coefficient["Coefficient Estimate"] = pd.Series(lreg.coef_)
print(lreg_coefficient)

```

In the next cell, we display the feature coefficients.

```

In [ ]: # plotting the coefficient score
fig, ax = plt.subplots(figsize=(20, 10))

ax.bar(
    lreg_coefficient["Columns"],
    lreg_coefficient["Coefficient Estimate"],
)

ax.spines["bottom"].set_position("zero")

plt.style.use("ggplot")
plt.suptitle(
    "Fig. 9: Coefficients Scores.", fontweight="bold", horizontalalignment="right"
)
plt.show()

```

```

In [ ]: # import ridge regression from `sklearn` library
from sklearn.linear_model import Ridge

# Train the model
ridgeR = Ridge(alpha=1)
ridgeR.fit(X_train, y_train)
y_pred = ridgeR.predict(X_test)

# calculate mean square error
mean_squared_error_ridge = np.mean((y_pred - y_test) ** 2)
print(mean_squared_error_ridge)

# get ridge coefficient and print them
ridge_coefficient = pd.DataFrame()
ridge_coefficient["Columns"] = X_train.columns
ridge_coefficient["Coefficient Estimate"] = pd.Series(ridgeR.coef_)
print(ridge_coefficient)

```

2. Lasso Regression

Lasso (Least Absolute Shrinkage and Selection Operator) is another regularization model that adds an L_1 -term to the objective function. It works by equating irrelevant parameters to zero. The loss function of a lasso regression is given by

$$L = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2 + \frac{\lambda}{2} \|w\|_1$$

Just like in ridge regression, λ controls the penalty's strength. Just like ridge regression above, when $\lambda = 0$, we have a linear regression equation. As the value of λ increases shrinkage to unimportant features and some will be equal to zero, the variance of the model reduces and bias increases.

In the cells below, we demonstrate how to apply lasso on our dataset.

```
In [ ]: steps = [("scalar", StandardScaler()), ("model", Lasso())

lasso_pipe = Pipeline(steps)
```

We apply cross-validation and print out the mean absolute error and the best alpha value.

```
In [ ]: cv = RepeatedKfold(n_splits=10, n_repeats=3, random_state=1)
model__alpha = np.linspace(0.1, 2, 21)
lasso = Lasso()
grid = dict()
grid["model__alpha"] = model__alpha
gscv = GridSearchCV(
    lasso_pipe, grid, scoring="neg_mean_absolute_error", cv=cv, n_jobs=-1
)
results = gscv.fit(X, y)
print("MAE: %.5f" % results.best_score_)
print("Config: %s" % results.best_params_)
```

We can now import the libraries and compute the model metric below.

```
In [ ]: from sklearn.linear_model import Lasso

# Train the model
lasso = Lasso(alpha=1)
lasso.fit(X_train, y_train)
y_pred1 = gscv.predict(X_test)

# Calculate Mean Squared Error
mean_squared_error = np.mean((y_pred1 - y_test) ** 2)
print("Mean squared error on test set", mean_squared_error)
lasso_coeff = pd.DataFrame()
lasso_coeff["Columns"] = X_train.columns
lasso_coeff["Coefficient Estimate"] = pd.Series(lasso.coef_)

print(lasso_coeff)
```

From the results above, we see the coefficients of features not relevant rounded down to zero.

2.2 Classification Metrics

In the next cells, we are going to discuss and demonstrate different classification metrics on the iris dataset.

```
In [ ]: from sklearn.datasets import load_iris
        from sklearn.model_selection import train_test_split

        data, target = load_iris(as_frame=True, return_X_y=True)

        X_train, X_test, y_train, y_test = train_test_split(
            data, target, shuffle=True, random_state=0, test_size=0.3
        )

        model = RandomForestClassifier(random_state=1)
        model.fit(X_train, y_train)
        target_predicted = model.predict(X_test)
```

Accuracy

We use accuracy as a baseline. Accuracy simply finds the proportion of classes we classified correctly, that is,

```
In [ ]: y_test == target_predicted
```

To get the accuracy of our model, we get the mean of the number of times we predicted correctly as shown below.

```
In [ ]: import numpy as np

        np.round(np.mean(y_test == target_predicted), 3)
```

This is implemented in `sklearn` using the `accuracy_score` as shown below.

```
In [ ]: from sklearn.metrics import accuracy_score

        accuracy = accuracy_score(y_test, target_predicted)
        print(f"Accuracy: {accuracy:.3f}")
```

Confusion Matrix

Accuracy might not be a good measure of a model performance, especially when the target variable is imbalanced. The most common class will always have more weight. To get the finer details of the model performance, it is recommended to construct a confusion matrix to independently determine what error is for each class. The cell below shows how to construct a confusion matrix

```
In [ ]: from sklearn.metrics import ConfusionMatrixDisplay

        _ = ConfusionMatrixDisplay.from_estimator(model, X_test, y_test)
        plt.suptitle(
            "Fig. 10: Confusion Matrix.", fontweight="bold", horizontalalignment="right"
        )
        plt.show()
```

The diagonal entries represent the correct predictions and the off-diagonal entries are the incorrect predictions. For more on confusion matrices, we ask students to read [A Review on Evaluation Metrics for Data Classification Evaluations](#).

3. Conclusion

In this lesson we discussed simulated annealing as one of the techniques for hyperparameter tuning and also introduced objective functions in machine learning. In the next lesson we are going to see how we can apply Bayesian hyperparameter tuning in a bitcoin trading strategy.

References

1. Ampatzis, Christos, and Dario Izzo. "Machine Learning Techniques for Approximation of Objective Functions in Trajectory Optimisation." Proceedings of the IJCAI-09 workshop on Artificial Intelligence in Space, 2009.
2. Botchkarev, Alexei. "Evaluating Performance of Regression Machine Learning Models Using Multiple Error Metrics in Azure Machine Learning Studio." 2018.
3. Hossin, Mohammad, and Md Nasir Sulaiman. "A Review on Evaluation Metrics for Data Classification Evaluations." *International Journal of Data Mining & Knowledge Management Process*, vol 5, no. 2, 2015.

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.